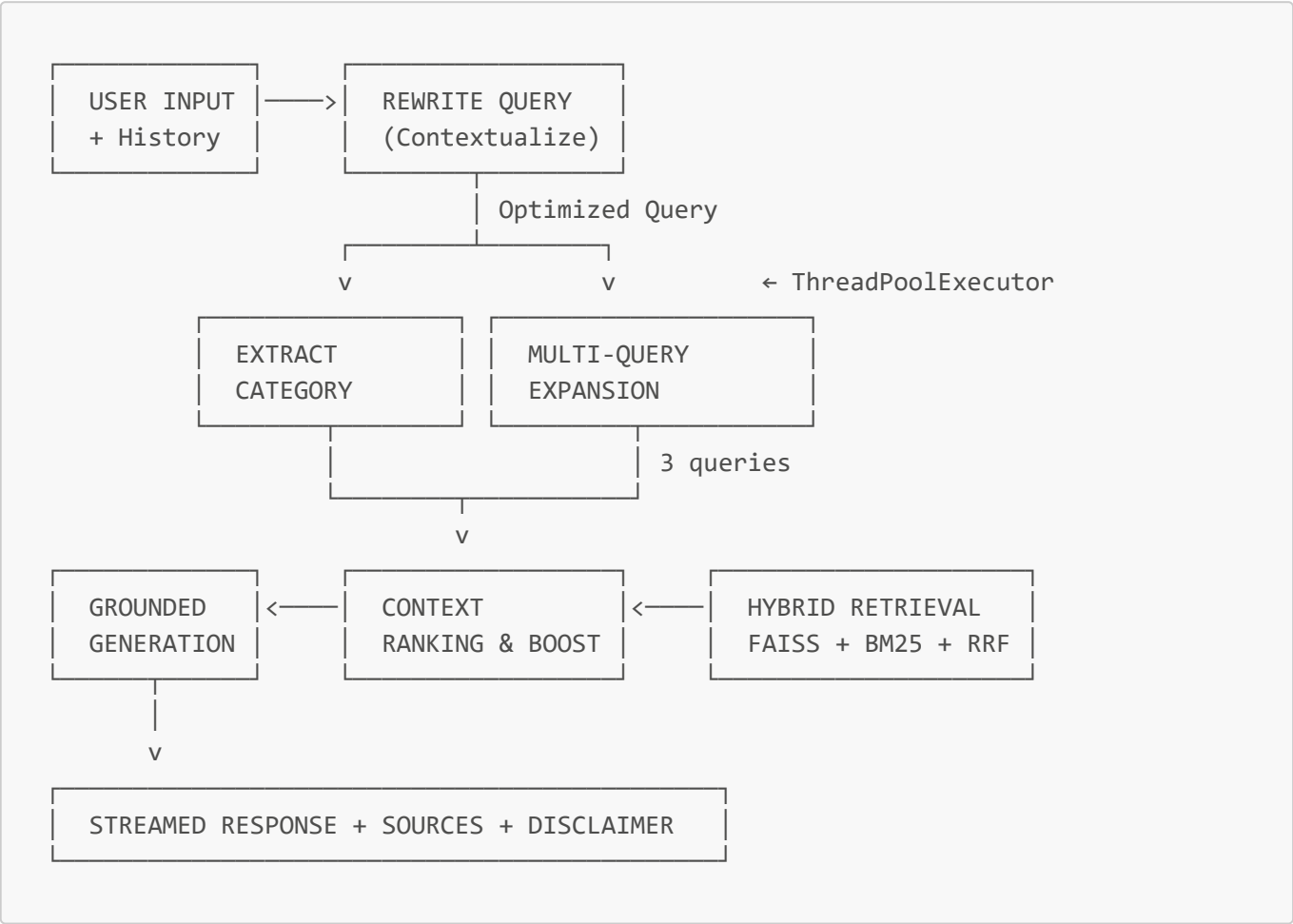


Luồng hoạt động kỹ thuật - DentalAI RAG System

Tài liệu mô tả chi tiết luồng dữ liệu từ lúc người dùng gửi câu hỏi đến khi nhận được câu trả lời, bao gồm các cơ chế chống ảo giác (hallucination) và chống ám thị chi tiết (detail suggestion bias).

Tổng quan luồng xử lý



Bước 1: Tiếp nhận đầu vào (User Input)

File: `src/chat/router.py` → `src/agent/chatbot.py`

Hệ thống nhận 2 đầu vào từ người dùng:

- **Câu hỏi hiện tại** (`user_question`): câu hỏi mới nhất của người dùng.
- **Lịch sử hội thoại** (`chat_history`): tối đa 8 cặp hỏi-đáp gần nhất, được lấy từ database theo `session_id`.

Lịch sử hội thoại đóng vai trò quan trọng trong Bước 2 — nếu không có lịch sử, hệ thống vẫn **luôn gọi LLM Rewrite** để chuẩn hóa câu hỏi (thêm chủ ngữ, từ khóa y khoa). Khi có lịch sử, hệ thống sẽ giải quyết được câu hỏi follow-up như "có đắt không?" hay "mất bao lâu?".

```
Request payload:
{
  "message": "có đất không?",
  "session_id": "abc-123"
}

→ Backend load 8 messages gần nhất từ DB
→ chat_history = [
  {"role": "user",      "content": "niềng răng là gì?"},
  {"role": "assistant", "content": "Niềng răng là phương pháp..."},
  {"role": "user",      "content": "có đất không?"}      ← câu hỏi mới
]
```

Bước 2: Biến đổi truy vấn (Query Transformation)

File: `src/agent/chatbot.py` → `rewrite_query()` + `extract_category()` + `expand_queries()`

Bước này gồm 3 giai đoạn — Rewrite chạy trước, sau đó Extract Category và Multi-Query Expansion chạy **song song** bằng `ThreadPoolExecutor(max_workers=2)`:

2a. Query Rewrite — Contextualization (tuần tự)

LLM **luôn** viết lại câu hỏi thành câu truy vấn tìm kiếm độc lập, đầy đủ ngữ cảnh — kể cả lượt chat đầu tiên khi chưa có lịch sử.

```
Input:  "có đất không?"
        + history chứa "niềng răng là gì?"

Output: "chi phí niềng răng tổng quan các loại phổ biến"
```

Các quy tắc rewrite quan trọng (định nghĩa tại `constants.py` → `REWRITE_USER_TEMPLATE`):

Quy tắc	Mục đích
Ghép chủ đề từ câu trước vào câu follow-up	Giải quyết đại từ / câu hỏi thiếu chủ ngữ
Không tự thêm tên thương hiệu, vị trí cụ thể	Chống ám thị chi tiết
Thêm "tổng quan" / "các loại phổ biến" cho câu hỏi chung	Ưu tiên tài liệu khái quát thay vì quảng cáo sản phẩm

Giải pháp "Chống Ám thị Chi tiết" (Detail Suggestion Bias): Vấn đề phát hiện: khi người dùng hỏi "quy trình niềng răng", hệ thống trả về quy trình riêng của Invisalign (vì dataset có nhiều bài về Invisalign) → câu trả lời bị thiên lệch. Giải pháp triển khai ở 3 tầng:

- 1. **Tầng Rewrite:** thêm từ khóa "tổng quan" để ưu tiên bài khái quát.
- 2. **Tầng Retrieval:** `_boost_overview()` đẩy bài có section "Tổng quan" lên đầu (Bước 4).

3. **Tầng Generation:** Rule 9 trong system prompt cấm LLM dùng thông tin thương hiệu để trả lời câu hỏi chung (Bước 5).

2b + 2c. Extract Category + Multi-Query Expansion (SONG SONG)

Sau khi có `rewritten_question`, hai tác vụ này chạy đồng thời bằng `ThreadPoolExecutor`:

2b. Entity Extraction — Phân loại bệnh lý:

LLM trích xuất tên bệnh/dịch vụ nha khoa chính từ câu hỏi để tiền lọc (pre-filter) kết quả tìm kiếm.

```
Input: "chi phí niềng răng tổng quan các loại phổ biến"
Output: ["niềng răng"]

Input: "cách chăm sóc răng miệng hàng ngày"
Output: None (câu hỏi quá chung, không filter)
```

2c. Multi-Query Expansion:

LLM sinh thêm 2 câu hỏi biến thể từ đồng nghĩa, tạo tổng cộng 3 truy vấn:

```
Query gốc: "chi phí niềng răng tổng quan"
Biến thể 1: "giá niềng răng các loại phổ biến hiện nay"
Biến thể 2: "bảng giá chỉnh nha bao nhiêu tiền"
```

Mục đích: tăng recall — nếu query gốc dùng từ "chi phí" nhưng tài liệu dùng từ "giá", biến thể sẽ bắt được.

Song song hóa:

```

Rewrite → [
    | extract_category() (Thread 1) |
    | expand_queries()   (Thread 2) |
] → search()
Tồng thời gian = max(Extract, Expand), không phải tổng
```

Fallback: Nếu Extract lỗi → `None` (bỏ qua filter). Nếu Expand lỗi → `[rewritten_question]` (chỉ dùng query gốc).

Danh mục và queries mở rộng được truyền trực tiếp vào `Retriever.search()` làm tham số.

Bước 3: Truy xuất hỗn hợp (Hybrid Retrieval)

File: `src/retriever/search.py` → `Retriever.search()`

Đây là bước cốt lõi của hệ thống RAG. `search()` nhận `expanded_queries` và `categories` từ bên ngoài (đã tính ở Bước 2), gồm 3 giai đoạn con:

3a. Category Pre-filtering

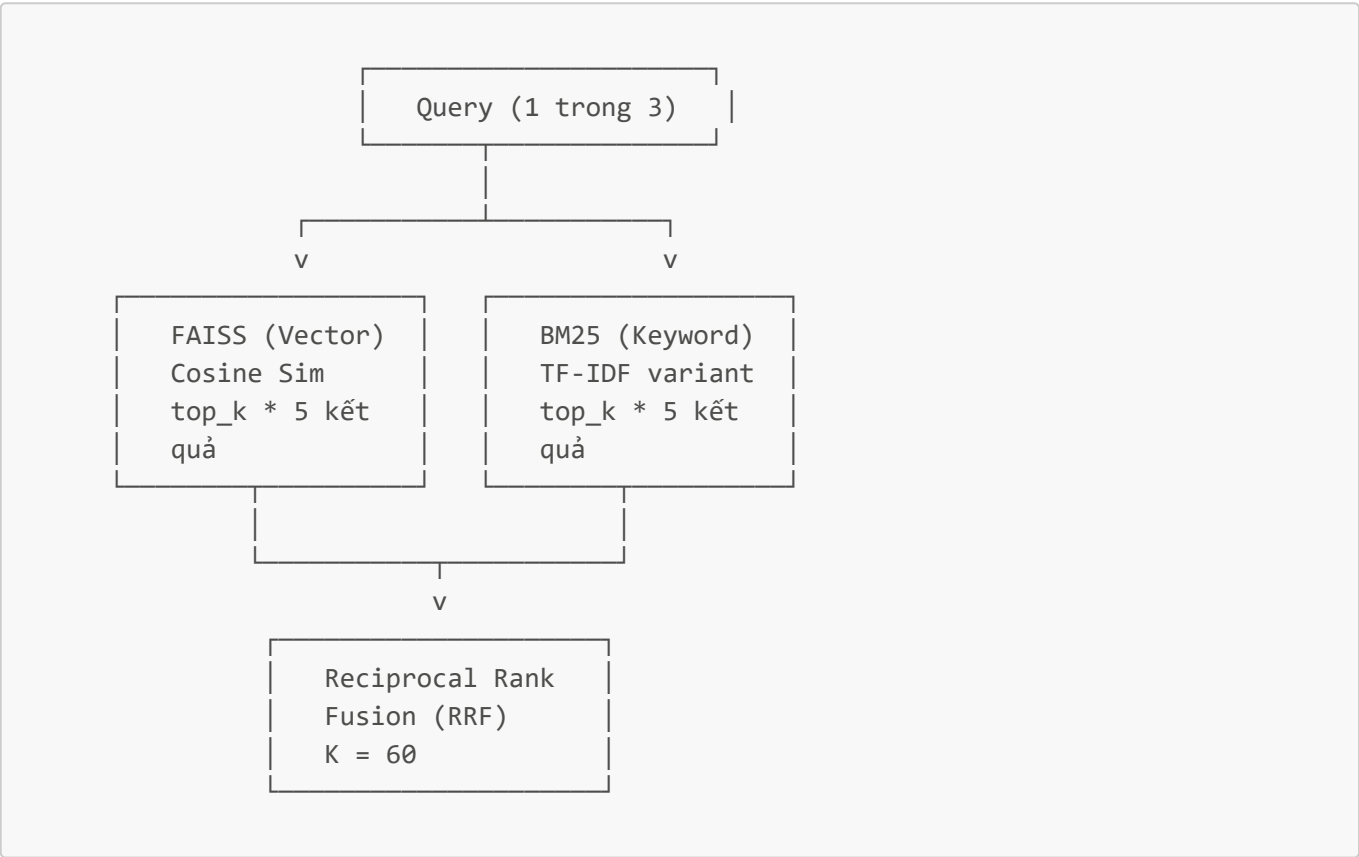
Nếu Bước 2b trả về category (VD: "niềng răng"), hệ thống lọc trước metadata:

```
762 tài liệu → chỉ giữ ~60 tài liệu có disease="niềng răng"
```

Giảm nhiễu đáng kể: tránh tình trạng bài về "sâu răng" xuất hiện khi hỏi về "niềng răng".

3b. Hybrid Scoring (cho mỗi query)

Mỗi query trong 3 query đều được chạy qua 2 kênh song song:



FAISS (Vector Search):

- Embedding engine mã hóa query thành vector (768-dim cho local, 1536-dim cho OpenAI).
- Tìm kiếm trên FAISS IndexFlatIP (Inner Product = Cosine Similarity do vector đã L2-normalized).
- Chuỗi embedding đã được làm giàu: "Tiêu đề: {title} | Mục: {section} | Tóm tắt: {summary} | Nội dung: {content}".

BM25 (Keyword Search):

- Tokenize tiếng Việt bằng Underthesea: "niềng răng" → "niềng_răng" (1 token).
- Corpus đã enriched: "{title} {section} {summary} {content}".
- Ưu điểm: bắt chính xác từ khóa y khoa mà vector search có thể bỏ sót.

Dynamic Weight — Trọng số động:

Loại câu hỏi	w_vector	w_bm25	Lý do
Chứa "chi phí", "bảng giá", "quy trình", "các bước",...	0.3	0.7	Cần khớp từ khóa chính xác
Câu hỏi thông thường	0.5	0.5	Cân bằng ngữ nghĩa và từ khóa

Reciprocal Rank Fusion (RRF):

```
RRF_score(doc) = w_vector / (K + rank_vector + 1)
                  + w_bm25   / (K + rank_bm25   + 1)

K = 60 (hằng số giảm tác động ranking quá cao)
```

Tài liệu xuất hiện ở cả FAISS lẫn BM25 được boost tự nhiên.

3c. Cross-Query Score Merging

Cộng điểm RRF từ 3 queries. Tài liệu được cả 3 biến thể tìm thấy sẽ có điểm cao nhất:

```
doc_A: query_1=0.012 + query_2=0.010 + query_3=0.011 = 0.033  ← top
doc_B: query_1=0.008 + query_2=0.000 + query_3=0.000 = 0.008  ← thấp hơn
```

Bước 4: Lọc và xếp hạng ngữ cảnh (Context Filtering & Ranking)

File: src/retriever/search.py → `_boost_overview()` File: src/agent/chatbot.py → `build_context()`

4a. Overview Boost

Sau khi xếp hạng, hệ thống đẩy các bài có tính tổng quan lên đầu danh sách:

```
Tín hiệu tổng quan: "tổng quan", "giới thiệu", "là gì", "các loại",
                    "tìm hiểu về", "quy trình chung"

Kiểm tra trong: title + section của mỗi tài liệu
```

Đây là tầng thứ 2 trong giải pháp **chống ám thị chi tiết**: đảm bảo bài "Niềng răng - Tổng quan" luôn đứng trước "Invisalign - Quy trình" trong danh sách kết quả.

4b. Context Building

Top-K tài liệu (`TOP_K` đọc từ `.env`, mặc định **K = 10**) được định dạng thành chuỗi ngữ cảnh:

```
Tiêu đề: Niềng răng
Mục: Tổng quan
Tóm tắt: Niềng răng là phương pháp chỉnh nha phổ biến...
Nội dung: (nội dung đầy đủ dạng markdown)
Nguồn: https://www.vinmec.com/...

---

Tiêu đề: Niềng răng
Mục: Chi phí
Tóm tắt: Chi phí niềng răng dao động từ 20-100 triệu...
Nội dung: ...
Nguồn: ...
```

Bước 5: Sinh câu trả lời có kiểm soát (Grounded Generation)

File: `src/agent/chatbot.py` → `answer_stream()` **File:** `src/lib/constants.py` → `AI_SYSTEM_INSTRUCTIONS`

LLM nhận 2 message:

- **System message:** 9 quy tắc cứng định nghĩa hành vi.
- **User message:** lịch sử hội thoại + ngữ cảnh nha khoa + câu hỏi gốc.

Các quy tắc chống ảo giác quan trọng

Rule	Tên	Hành vi
Rule 2	Strict Grounding	CHỈ trả lời dựa trên ngữ cảnh được cung cấp, KHÔNG dùng kiến thức tự có
Rule 3	Từ chối khi không có dữ liệu	Nếu ngữ cảnh không liên quan → trả đúng 1 câu từ chối cố định
Rule 4	Giới hạn chuyên môn	Chỉ trả lời câu hỏi nha khoa, từ chối câu hỏi ngoài phạm vi
Rule 9	Chống ám thị chi tiết	Cấm dùng thông tin thương hiệu cụ thể để trả lời câu hỏi chung

Rule 9 chi tiết — Giải pháp "Tràn kiến thức" (Knowledge Overflow):

Vấn đề: Khi ngữ cảnh chỉ chứa thông tin của 1 thương hiệu (VD: Invisalign), LLM có xu hướng trình bày quy trình riêng của thương hiệu đó như quy trình chung của toàn ngành.

Giải pháp trong Rule 9:

- Nếu câu hỏi chung → **BẮT BUỘC** tổng hợp câu trả lời khái quát.
- Nếu ngữ cảnh chỉ có 1 hãng → phải ghi rõ: "Theo quy trình của [Tên Hãng], các bước gồm..."
- **KHÔNG** mang tên sản phẩm, tên giai đoạn bệnh cụ thể vào nếu người dùng không hỏi.

Streaming Response

Câu trả lời được stream từng chunk qua Server-Sent Events (SSE):

1. Stream nội dung text

→ hiển thị real-time trên UI

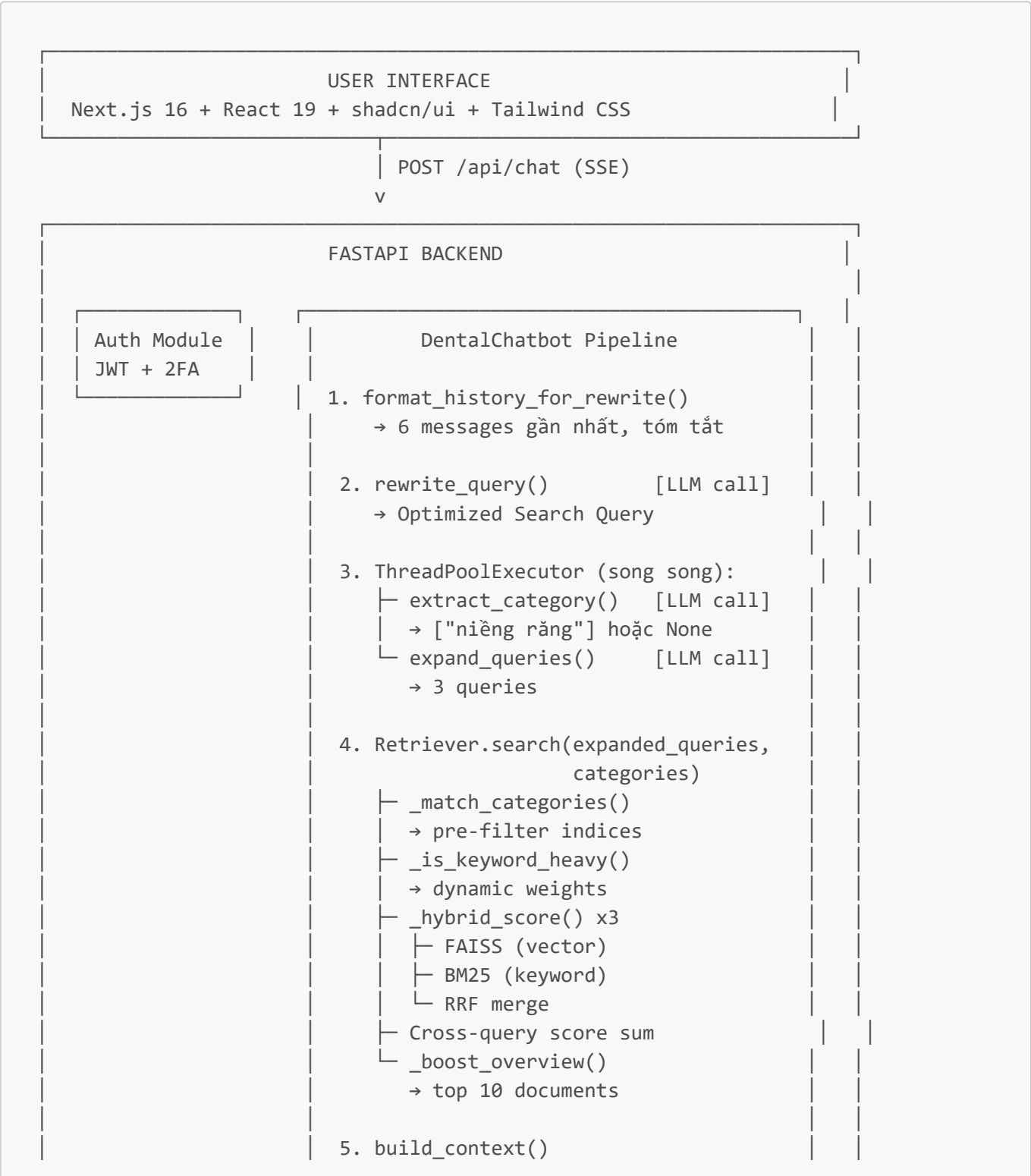
2. Append disclaimer

→ "Thông tin chỉ mang tính tham khảo..."

3. Yield metadata object

→ { sources: [...], rewritten_query: "..." }

Sơ đồ tổng hợp (End-to-End)





Tổng kết các LLM call trong 1 request

Thứ tự	Hàm	Mục đích	Temperature	Ghi chú
1	rewrite_query()	Viết lại câu hỏi + thêm ngữ cảnh	0.0 (STRICT)	Luôn chạy (kể cả câu đầu tiên)
2a	extract_category()	Trích xuất bệnh/dịch vụ để pre-filter	0.0 (STRICT)	Song song với 2b
2b	expand_queries()	Sinh 2 biến thể từ đồng nghĩa	0.5	Song song với 2a
3	answer_stream()	Sinh câu trả lời cuối cùng (stream)	0.3 (NORMAL)	

Tổng cộng: **4 LLM calls** cho mỗi câu hỏi (trong đó 2a và 2b chạy song song bằng `ThreadPoolExecutor`).